

TP : Mini-projet intégrateur – Application de gestion d'une bibliothèque

Objectifs

À la fin de ce mini-projet, vous serez capable de :

- Développer une application web complète en PHP.
- Utiliser **PDO** pour communiquer avec une base de données MySQL.
- Mettre en œuvre un système d'authentification.
- Réaliser les opérations **CRUD** (Créer, Lire, Modifier, Supprimer).
- Mettre en place une recherche multicritère.
- Gérer la pagination des résultats.
- Organiser le projet selon une architecture simple.
- Sécuriser l'application (sessions, requêtes préparées, validation des données).

Présentation du projet

Vous devez développer une application permettant de gérer une bibliothèque universitaire.

L'application doit permettre :

- la connexion des administrateurs ;
- la gestion des livres ;
- la gestion des étudiants ;
- la gestion des emprunts ;
- la recherche d'informations ;
- la pagination des listes ;
- la déconnexion.

Toutes les opérations sur la base de données doivent être réalisées avec **PDO**.

Cahier des charges

Authentification

L'utilisateur doit pouvoir :

- se connecter ;
- accéder uniquement aux pages protégées ;
- se déconnecter.

Les mots de passe doivent être stockés avec `password_hash()` et vérifiés avec `password_verify()`.

Gestion des livres

Le système doit permettre :

- Ajouter un livre
- Modifier un livre
- Supprimer un livre
- Afficher la liste des livres
- Rechercher un livre

Informations

- Titre
- Auteur
- [ISBN](#)
- Année
- Quantité disponible

Gestion des étudiants

Le système doit permettre :

- Ajouter
- Modifier
- Supprimer
- Afficher
- Rechercher

Informations

- Nom
- Prénom
- Email
- Téléphone
- Filière

Gestion des emprunts

Le système doit permettre :

- Enregistrer un emprunt
- Retourner un livre
- Consulter les emprunts
- Rechercher un emprunt

Informations

- Étudiant
- Livre
- Date d'emprunt
- Date de retour prévue
- Date de retour effective
- Statut (En cours, Retourné)

Base de données

Création de la base

```
CREATE DATABASE bibliotheque;

USE bibliotheque;
```

Table utilisateurs

```
CREATE TABLE utilisateurs
(
id INT AUTO_INCREMENT PRIMARY KEY,

nom VARCHAR(50),

email VARCHAR(100) UNIQUE,

mot_de_passe VARCHAR(255)
);
```

Table livres

```
CREATE TABLE livres
(
id INT AUTO_INCREMENT PRIMARY KEY,

titre VARCHAR(150),

auteur VARCHAR(100),

isbn VARCHAR(20),

annee INT,
```

```
quantite INT
);
```

Table étudiants

```
CREATE TABLE etudiants
(
id INT AUTO_INCREMENT PRIMARY KEY,

nom VARCHAR(50),

prenom VARCHAR(50),

email VARCHAR(100),

telephone VARCHAR(30),

filiere VARCHAR(100)
);
```

Table emprunts

```
CREATE TABLE emprunts
(
id INT AUTO_INCREMENT PRIMARY KEY,

id_livre INT,

id_etudiant INT,

date_emprunt DATE,

date_retour_prevue DATE,

date_retour DATE NULL,

statut VARCHAR(20),

FOREIGN KEY(id_livre)
REFERENCES livres(id),

FOREIGN KEY(id_etudiant)
REFERENCES etudiants(id)
);
```

Architecture du projet

```
bibliotheque/

config/
    config.php
    database.php
```

```
controllers/  
    AuthController.php  
    LivreController.php  
    EtudiantController.php  
    EmpruntController.php  
  
models/  
    Utilisateur.php  
    Livre.php  
    Etudiant.php  
    Emprunt.php  
  
views/  
  
    auth/  
        connexion.php  
  
    livres/  
        liste.php  
        ajouter.php  
        modifier.php  
  
    etudiants/  
        liste.php  
        ajouter.php  
        modifier.php  
  
    emprunts/  
        liste.php  
        ajouter.php  
        modifier.php  
  
assets/  
  
    css/  
    js/  
  
index.php  
login.php  
logout.php
```

Fonctionnalités à développer

1. Connexion

- Formulaire de connexion
- Vérification des identifiants
- Création de la session
- Protection des pages

2. Déconnexion

- Destruction de la session

- Redirection vers la connexion

3. CRUD Livres

Créer les pages :

- liste.php
- ajouter.php
- modifier.php
- supprimer.php

4. CRUD Étudiants

Créer les pages :

- liste.php
- ajouter.php
- modifier.php
- supprimer.php

5. CRUD Emprunts

Créer les pages :

- liste.php
- ajouter.php
- modifier.php
- supprimer.php

6. Recherche

Ajouter une barre de recherche.

Exemple :

Rechercher un livre...

La recherche doit fonctionner sur :

- titre
- auteur

- ISBN

Exemple SQL :

```
SELECT *  
FROM livres  
WHERE titre LIKE ?  
OR auteur LIKE ?  
OR isbn LIKE ?;
```

7. Pagination

Afficher **10 résultats par page**.

Exemple SQL

```
SELECT *  
FROM livres  
LIMIT 10 OFFSET 0;
```

Page 2

```
LIMIT 10 OFFSET 10;
```

8. Interface utilisateur

Le menu doit contenir :

Accueil
Livres
Étudiants
Emprunts
Déconnexion

9. Sécurité

L'application doit utiliser :

- PDO
- Requêtes préparées
- Sessions

- Validation
- Filtrage
- `htmlspecialchars()`
- `password_hash()`
- `password_verify()`

Travail demandé

Développer l'application complète.

Toutes les fonctionnalités doivent être opérationnelles.

Barème d'évaluation

Critère	Points
Connexion et authentification	10
Gestion des livres (CRUD)	20
Gestion des étudiants (CRUD)	20
Gestion des emprunts (CRUD)	20
Recherche	10
Pagination	5
Utilisation correcte de PDO	10
Qualité de la présentation, du code et sécurité	5
Total	100

Exercices complémentaires

Exercice 1

Ajouter une recherche multicritère sur les étudiants.

Exercice 2

Afficher uniquement les livres disponibles.

Exercice 3

Afficher la liste des emprunts en retard.

Exercice 4

Afficher le nombre total de livres empruntés.

Exercice 5

Empêcher l'emprunt d'un livre lorsque la quantité disponible est égale à **0**.

Bonus (facultatif)

- Tableau de bord avec statistiques (nombre de livres, étudiants, emprunts en cours).
- Gestion des catégories de livres.
- Upload de la couverture des livres.
- Tri des listes par titre, auteur ou date.
- Export des listes au format PDF ou Excel.
- Interface responsive avec **Bootstrap**.
- Journalisation des actions (ajout, modification, suppression).

Bonnes pratiques

- Utiliser une connexion PDO unique centralisée dans `database.php`.
- Employer des **requêtes préparées** pour toutes les opérations SQL.
- Valider et filtrer les données saisies avant de les enregistrer.
- Protéger toutes les pages par une authentification basée sur les sessions.
- Structurer le projet en séparant la configuration, les modèles, les contrôleurs et les vues.
- Commenter les parties importantes du code et adopter une nomenclature cohérente.

Résumé

Dans ce mini-projet, vous mettez en pratique l'ensemble des notions étudiées dans les chapitres précédents :

- **PHP** et la programmation côté serveur.
- **Formulaires HTML** et traitement des données.
- **Sessions** et **cookies**.
- **MySQL** et le langage SQL.
- **PDO** et les requêtes préparées.
- **Authentification** et **sécurisation** des applications.
- **Architecture** d'une application PHP.
- **CRUD**, **recherche** et **pagination**.

Ce projet constitue une excellente synthèse du cours et permet de développer une application web complète, sécurisée et maintenable en utilisant les bonnes pratiques de développement PHP avec **PDO**.